

Joshua Coetzer

09HA2603995

HCGDP1181 (Creative Project: Game Production)

Babble Bean Studios

Proposal for breeding mechanic in Cryptid Sanctuary

The approach to designing the system

1. Starting from the existing systems:

- I have studied the existing project and taken a look at how the scripts and objects work together, so that I can understand how to extend it as well as stick to the existing conventions.
- The conventions that I noticed are that each room's script inherits from a base Room script, that the data for the cryptids is stored in the Trait and Stat scripts, and that the Manager scripts are singletons.
- The new BreedingRoom will follow the same patterns as the other Rooms.

2. Requirements and which scripts will handle them:

- Exactly two creatures (no more, no less) will be handled by [BreedingRoom.cs](#)
- Inheritance from both parents will be handled by [Genetics.cs](#)
- The passed down visual trait will be handled by [CreatureGenes.cs](#) which will pass down colour and scale
- Mutation that's not from either parent will be handled by a [MutationTable.cs](#)

3. Breaking down each script:

Script	Kind of script	Where it lives	What it does
BreedRoom.cs	MonoBehaviour that inherits from Room	On new Breeding Room prefab	Counts the creatures, runs timer, triggers birth
CreatureGenes.cs	MonoBehaviour	Added to existing cryptid prefabs	Stores colour and scale and applies to model
Genetics.cs	Normal C# class	Only code	Takes 2 parents genes, works out baby genes
MutationTable.cs	ScriptableObject	Create .asset file	Hold tunable data like chance & colours

4. Things that help other non-programmer team members:

- Mutation is in a ScriptableObject, so no need to change code to rebalance breeding
- Genetics math in plain C# class with no Unity dependency makes unit-testing easier

5. Keep it simple, stupid:

- I chose simple inheritance rules like colour and scale because they'll be easy to implement and debug, and are things that are obvious from both parents
- I also opted to leave out complex features like family trees or dominant/recessive genes, since even though they're interesting, I've kept scope in mind.

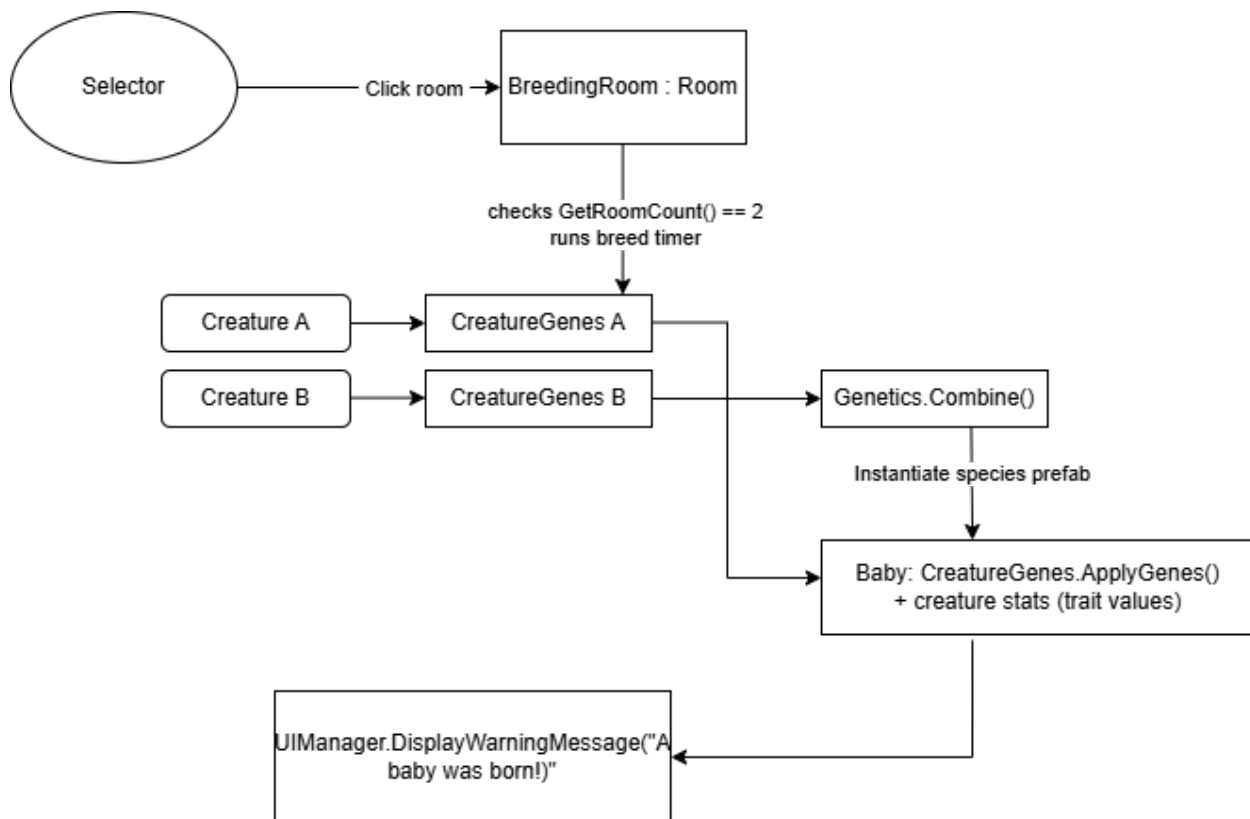
6. One piece at a time:

- I will be building it in small milestones, testing each one before moving to the next.
- I'll commit each milestone to Git so that if anything goes wrong, I can fall back to an earlier working build

7. Designed for the player:

- Breeding should keep in line with the cozy Tamagotchi style/concept
- Colour and scale are noticeable at a glance.
- Will include UI messages just in case player misses it

Scripts diagram



How the scripts work (layman's explanation)

Each creature carries a small "DNA card" listing its colour and size. When exactly two creatures are left alone in the breeding room for a while, the game reads both cards and deals out a new one. The colour and scale is a mix of both parents, and every other gene comes from one parent or the other. Now and then a gene "mutates" and picks something from a list neither parent had. A new creature appears with that card and looks and behaves to match it.

Additional resources

Pokemon breeding:

In Pokemon, two compatible Pokemon left together can produce an Egg. The baby is the same species as one parent, and some of its hidden stats (IVs) are copied from either parent at random while the rest are random (Bulbapedia, n.d.). This supports two decisions, being my baby will spawn from one parent's species prefab, and each stat gene will come from one parent or the other. The random IVs are also a precedent for my small mutation chance.

Bulbapedia (n.d.) Pokemon breeding. Available at:

https://bulbapedia.bulbagarden.net/wiki/Pok%C3%A9mon_breeding

Game Freak (2022) Pokemon Scarlet and Violet [Video game]. Kyoto: Nintendo / The Pokemon Company.

Unity ScriptableObject:

A ScriptableObject is a Unity data asset that lives in the project rather than on a GameObject in the scene (Unity Technologies, n.d. a). My MutationTable will be a ScriptableObject holding the mutation chance, colour pool and scale range. A designer can then rebalance breeding in the Inspector without touching code, and several breeding rooms can share one set of settings.

Unity Technologies (n.d. a) ScriptableObject. Available at:

<https://docs.unity3d.com/6000.0/Documentation/Manual/class-ScriptableObject.html>

Component pattern:

Nystrom (2014a) describes the Component pattern as splitting an object's behaviour into separate, self-contained pieces rather than one large class. The existing project already has each creature built from separate stat components. I'll follow the same convention by adding CreatureGenes as its own component instead of expanding

Creature.cs, which keeps genetics isolated and leaves the studio's existing code unchanged.

Nystrom, R. (2014a) Game Programming Patterns: Component. Available at:

<https://gameprogrammingpatterns.com/component.html>

Type Object pattern:

The Type Object pattern defines variations of a thing as data instead of writing a new class for each one (Nystrom, 2014b). I'll apply it to mutations where each possible mutation is an entry in the MutationTable asset and not a new script. Adding a new mutation then means adding data instead of code which keeps the system easy to extend.

Nystrom, R. (2014b) Game Programming Patterns: Type Object. Available at:

<https://gameprogrammingpatterns.com/type-object.html>

Unity Test Framework:

The Unity Test Framework lets you write automated tests that run inside the Unity Editor (Unity Technologies, n.d. c). Because my Genetics class is plain C# with no scene dependencies, I can write tests confirming that a baby always inherits from both parents and that mutation stays within its set chance.

Unity Technologies (n.d. c) Unity Test Framework scripting reference. Available at:

<https://docs.unity3d.com/Packages/com.unity.test-framework@1.6/manual/index.html>

Timeline

Dates	Milestone	Completed when ...
25 Sep	FA1: Proposal, M1: Initial creature genes	Proposal submitted, genes change creature's colour and scale
28 Sep - 4 Oct	M2: Breeding room	Room prefab has baked NavMesh and MovePoint. The exactly-2 rule and breeding timer work. 1 or 3+ creatures blocked.
5-11 Oct	M3: Offspring spawning	Baby of one of parent's species spawns, parented to room, with genes applied
12-18 Oct	M4: Inheritance	Genetics blends colours and picks stats from parents. Unit tests pass.
19-25 Oct	M5: Mutation	MutationTable exists and mutations occasionally product new traits
26 Oct - 1 Nov	M6: Feedback and UI	"Baby born" and "Too many creatures" message appears. Genes shown in creature info panel
2 - 8 Nov	Testing and bug fixing	Done playtesting, edge cases (e.g. parent dies during breeding)
9-13 Nov	Code freeze and submission preparation	Repo is tidy with folders and names consistent. Fresh clone opens cleanly.
16-17 Nov	Summative submission	PDF with GitHub link & completed checklist uploaded

(M<x> = Milestone <x>)